Recovery Contracts for Post-Quantum Storage Migration

Ishan Teredesai Tushar Rahangdale
Department of Information Technology
Yeshwantrao Chavan College of Engineering, Nagpur, India
teredesaishan@gmail.com tusharrahangadle@gmail.com

***Abstract*—Migrating a storage service to a new cryptographic profile changes more than its active encryption path: retained backups and interrupted key updates can still depend on an older key generation. We argue that migration tooling should expose two separate contracts: preserving key references needed for recovery, and covering every retained restore point with an approved protection profile. We specify a small transition system and compare retirement rules that inspect intended writes, committed live records, or the full modeled recovery set. Exhaustive exploration of eighteen bounded configurations finds live-data or backup failures under the first two rules and no missing-key state under the recovery-set rule. A separate SQLite/AES-GCM harness executes 126 deterministic cases, including 108 process terminations. The recovery-set rule restores all records and backups in its 42 cases; the other rules expose distinct commit-ordering and backup-retention failures. These results concern lifecycle correctness in an abstraction and a synthetic harness; they do not establish post-quantum security or production reliability. We contribute an executable contract, reproducible counterexamples, and a deployment-oriented validation agenda for critical systems.**

***Index Terms*—post-quantum migration, key management, backup recovery, crash consistency, software assurance**

I. Introduction

Critical services must preserve access to historical data while changing the cryptography that protects it. A hospital may need an older patient-record snapshot during an incident even after its live database has moved to a new key generation. If migration retires the key required by that snapshot, successful encryption of current records says little about the ability to recover the service. This example motivates our scope; we do not evaluate a hospital deployment or use patient data.

NIST's ML-KEM standard provides a post-quantum key-establishment primitive [1], and its crypto-agility guidance treats migration as an operational concern [2]. Neither choosing a primitive nor changing a middleware configuration by itself specifies which historical envelopes, restore tools, and keys must remain usable. In an envelope design, a data encryption key (DEK) protects the record, and a separate key or key-establishment construction protects the DEK. Replacing the latter need not rewrite the bulk ciphertext. The persistence and retention of the old envelope therefore matter.

This is an instance of established key-management and crash-consistency problems, rather than a new cryptographic construction. NIST key-management guidance covers key inventory, backup, recovery, and destruction [3]. AWS KMS explicitly retains earlier key material for decryption during ordinary rotation [4]; MongoDB documents DEK rewrapping and warns against deleting keys still in use [5]. Prior systems research shows that application update protocols depend on persistence ordering [6]. Our contribution is to make a small part of this guidance executable for migration reviews: which references a retirement decision must inspect, and what a migration-completion indicator must actually mean.

We study the question: *When may storage middleware retire an old key generation while preserving its declared recovery obligations?* We distinguish a durable reference from an intended update, and a live database from its retained restore set. We provide (1) a contract and preservation argument for a finite-state abstraction, (2) complete exploration of selected small instances with shortest counterexample traces, and (3) process-crash experiments in a synthetic encrypted-storage harness. The artifact makes deliberately weak policies observable; it does not assert that existing KMS products implement them.

II. Scope and Recovery Contracts

A. System boundary and assumptions

We model a trusted migration controller, an inventory of available key generations, committed live record envelopes, and one retained full snapshot. A generation identifies a key and a protection-profile label. In a real migration, that label would identify an authenticated envelope format, algorithm suite, and compatible recovery implementation. The model abstracts these details as integers: generation zero is labeled legacy and later generations target. A label is a policy annotation, not evidence of the security of its algorithm.

The failure of interest is interruption between durable operations, including the interruption of a database transaction while a separate key inventory has already changed. The controller and inventory are assumed honest. Key compromise, forged metadata, broken cryptographic primitives, KMS outages, malicious rollback, and side channels are outside the checked model. In particular, retaining an old decryption key may conflict with a compromise-response policy. Such a conflict requires an explicit decision about preserving or abandoning affected restore points; our contract does not resolve it automatically.

B. Reference preservation

Let a state be $S = (a, L, B, P, K)$, where a is the active generation, L is the vector of committed live references, B is either one snapshot vector or absent, P contains each record's volatile pending target or $\perp$, and K is the set of durably available generations. Define

$$R(S) = \{L_i\} \cup \{B_i : B \text{ is present}\}, \tag{1}$$

$$I_{\text{ref}}(S): \quad R(S) \subseteq K. \tag{2}$$

The invariant is deliberately narrower than successful service recovery: it states that the modeled references have keys. It does not check parser compatibility, authorization, replica availability, or the integrity of a real backup. Those become additional obligations at deployment time.

Our proposed retirement rule permits removing k only if

$$k \neq a \ \wedge \ k \notin R(S) \ \wedge \ k \notin \{P_i : P_i \neq \perp\}. \tag{3}$$

The last clause also avoids invalidating prepared work directed to an older generation during another activation. A database commit and a separate key deletion are not treated as one atomic transaction. A deployment must serialize the reference check with changes that could introduce a new dependency, or use a versioned retirement protocol with an equivalent effect.

C. Profile coverage

Reference preservation can hold while a retained snapshot still uses the legacy profile. Let Π be the approved target profiles. A separate inventory property is

$$I_{\text{profile}}(S): \quad \forall r \in R(S), \ \text{profile}(r) \in \Pi. \tag{4}$$

In the experiment, this is simply $r > 0$ for every retained reference. A dashboard that inspects only L can report completion while B remains legacy. Both properties are necessary evidence for a declared migration of the modeled recovery set; together they still do not prove cryptographic security. Rewrapping present copies cannot undo disclosure of an older envelope already captured by an adversary.

III. An Executable Transition Model

The initial state has all live records and a retained snapshot at generation zero, $K = \{0\}$, and no pending work. Activation provisions the next generation durably before making it active. Prepare uses an available source key to build a volatile replacement envelope. Commit atomically changes one live reference only if its target key is available. A crash discards all pending work. Snapshot copies committed live references when no snapshot is retained; release removes the snapshot from the declared obligations. Release represents an explicit retention decision, not a claim about physical erasure. The model excludes ordinary content updates, replicas, and multiple retained snapshots.

We compare three retirement rules (Table I). *Intent-only* mistakes pending replacements for committed state and ignores the snapshot. *Live-only* inspects committed live references but ignores the snapshot. *Recovery-set* applies (3). The first two are deliberately weak controls chosen to isolate the two dependencies.

TABLE I
References that prevent retirement of a non-active key

Rule	References inspected
Intent-only	Pending target when present; otherwise the live reference.
Live-only	Committed live references only.
Recovery-set	Committed live references, retained snapshot, and pending targets.

TABLE II
Complete reachable-state exploration: three records, three generations

Rule	States	Missing live key	Missing backup key
Intent-only	3,682	1,814	1,766
Live-only	1,996	0	403
Recovery-set	1,495	0	0

A. Preservation argument

Assume I_{ref} holds initially and transitions are serialized as specified. Activation adds a key. Prepare and crash do not change durable references. Commit points to an available target. Snapshot copies valid live references, and release removes obligations. Finally, (3) prevents retirement of a referenced generation. Each transition therefore preserves I_{ref}. This is a reasoning argument for the stated abstraction, not a machine-checked cryptographic proof or a proof of a distributed implementation. The artifact also checks preservation over a small structurally valid state universe, including states that its reachability traversal does not visit.

B. Exploration procedure

A deterministic breadth-first traversal enumerates successors until the queue is empty, retaining parents for shortest traces. It does not stop at a selected trace depth or state-count threshold. We vary the number of live records over $\{1, 2, 3\}$ and generations over $\{2, 3\}$, with each of the three policies: eighteen configurations. Snapshot release and creation may repeat, but states are deduplicated. These bounds capture interleaved record migration and a second activation; they do not model a large deployment. Counts refer to reachable abstract states, not randomly sampled incidents or failure probabilities.

IV. Evaluation

A. Bounded model results

Table II reports the largest configuration. Intent-only permits losing the key of a committed live record. Live-only preserves live references but admits states in which a retained backup has no key. Recovery-set has no missing-key state in this configuration, nor in the other five configurations for that rule: 1,890 states and 7,232 generated transitions in total, counting each configuration separately. The two failure columns can overlap and must not be added as disjoint counts.

The smallest examples need only one record and two generations. For intent-only, activate generation 1, prepare a

rewrap, and retire generation 0. The durable live reference remains 0; a crash discards the only pending replacement. For live-only, activate, prepare, commit, then retire generation 0. The live reference is now 1, but the retained snapshot still requires 0. Recovery-set rejects that retirement until the old snapshot obligation is released or otherwise resolved.

Profile coverage fails independently of key loss. After activation and live commit, both keys may remain available while the snapshot is still legacy. Such states occur under every rule, including 293 states in the largest recovery-set configuration. This demonstrates why reference preservation and profile coverage must be reported separately; it is not a measured incidence rate.

B. Process-crash harness

We implement a separate synthetic harness using SQLite 3.53.1, Python 3.12.14, and `cryptography` 50.0.2 on Linux x86-64. Each record receives a fresh 256-bit DEK. AES-256-GCM encrypts both the payload and the wrapped DEK, with separate random 96-bit nonces and context-bound associated data. A local JSON key inventory is updated using file synchronization and atomic replacement. The database uses rollback journaling and `synchronous=FULL`. A retained database snapshot is created before rotation. SQLite's transaction guarantees [7] do not make the independent key inventory part of the same transaction.

This harness tests persistence ordering with actual ciphertext and restores. It does not implement ML-KEM, a hybrid KEM, a remote KMS, or a QRNG. Its epochs represent successive wrapping keys. The model's legacy/target labels are therefore not claims that the harness measures post-quantum protection.

The controller first provisions a new key, prepares all replacement envelopes, attempts retirement under the selected rule, updates the database transaction, commits, and attempts retirement again. We terminate its child process with `os._exit(93)` after provisioning, after preparation, after the first retirement attempt, before database commit, after commit, or after the final retirement attempt. A seventh case completes normally. The parent then reopens both databases and checks every recovered plaintext against the generated input. The host kernel and filesystem remain running throughout.

We test one and eight records, each at 512 B, 16 KiB, and 256 KiB: six datasets. Crossing these with three rules and seven termination/completion cases gives 126 cases, of which 108 terminate the child. Each configuration runs once. The schedules are deterministic; independent random keys and nonces vary ciphertext bytes without changing the expected dependency failures. These are case-coverage outcomes, not statistical reliability estimates. No latency or throughput measurements are reported.

C. Observed failures

Table III agrees with the two abstract counterexamples. Intent-only loses live data at the cut after its premature retirement and at the cut before commit: two cuts across six datasets. Its backups fail at five cuts/completion points.

TABLE III
CRASH-HARNESS OUTCOMES, 42 CASES PER RULE

Rule	Live failure	Backup failure	Both restored
Intent-only	12	30	12
Live-only	0	12	30
Recovery-set	0	0	42

Live-only loses backups after its final retirement and after normal completion, while live restoration succeeds. Thus an apparently successful migration can leave recovery obligations broken even without an interruption. Recovery-set retains the old key while the snapshot remains required and restores both copies in all 42 cases. Every reported failure is a missing wrapping-key dependency; the harness treats an authentication failure, unexpected row count, or plaintext mismatch as an experiment error.

The finite model and the harness share the contract but exercise different mechanisms. The model explores interleavings of individual record operations; the harness migrates each dataset in one SQL transaction and interrupts a real process. Agreement increases confidence in these particular failure mechanisms. It does not constitute independent validation of the model by an existing production system.

V. Deployment Implications for Critical Systems

The operational unit of migration should be a declared *recovery set*. Its inventory should include retained snapshots, replicas, exports, and log segments as applicable, together with the key and decoder dependencies needed to restore them. Our single-snapshot implementation is a minimal instance of this broader obligation. In a healthcare deployment, evidence for retirement would include a test restore of the oldest retained protected record and the authorization path used during an outage. These are proposed acceptance tests, not observations from a hospital.

Two progress measures should be exposed: live-profile coverage and retained-copy profile coverage. A third indicator should identify blocked retirement and explain the dependency. Keeping an old key can preserve availability while leaving an older protection profile in the restore set; deleting it can eliminate recoverability. Operators need an explicit migration or retention decision rather than a single percentage labeled complete.

The guard also has an operational cost. Enumerating all references per retirement is linear in the declared inventory; reference counts or epoch manifests can reduce the lookup cost but introduce update and reconciliation obligations. We have not measured these costs. A production evaluation should vary snapshot retention, concurrent migration workers, inventory size, KMS latency/outage behavior, and restore volume, measuring both retained-key occupancy and recovery time. Where old envelopes can be safely rewrapped, their update protocol must itself be tested. Where snapshots are immutable, a documented copy-and-verify or expiry policy is required.

VI. Limitations and Research Agenda

Abstraction validity. One retained snapshot and at most three records omit distributed consistency, unknown replicas, ordinary writes/deletes, stale authorization, and corrupted metadata. Atomic steps assume correct serialization between inventory inspection and retirement. A stale scan can invalidate the argument. An implementation must protect this check from races and demonstrate that its inventory is complete.

Failure coverage. Process termination leaves the operating system alive. We do not emulate power loss, torn writes, a failed storage device, or remote-KMS transactions. File-system persistence differences matter [6]; wider crash testing is required before making a platform guarantee. Dataset sizes test correctness paths only and establish no scaling or performance claim.

Security scope. Neither experiment proves confidentiality, hybrid-combiner security, nonce safety over a deployment lifetime, or resistance to compromised middleware. Production KEM integration must follow appropriate standards and protocol analysis [8]. Profile labels must be tied to verified implementations and authenticated metadata. A retained copy that was already exposed under a vulnerable scheme remains exposed regardless of subsequent rewrapping.

Next evaluation. The immediate extension is to apply the contract to a real middleware/KMS integration with multiple retained backups and concurrent workers. The primary outcomes should be actual restoration, safe retirement under interruption, and auditable profile coverage. Cryptographic benchmarks and hardware-entropy studies are separate tasks and should not substitute for this lifecycle evidence.

VII. Conclusion

Key retirement is a decision about durable dependencies, including the dependencies of retained backups. A small executable model and a process-crash harness expose two ways to lose those dependencies. The recovery-set guard preserves the stated invariant in the abstraction and restores every tested dataset in the synthetic harness. For post-quantum migration, that result must be complemented by separate evidence that all retained copies use an approved profile. These contracts provide a concrete starting point for testing migration tooling in critical systems, with the remaining implementation and security assumptions made explicit.

Artifact and Related Work Disclosure

The accompanying artifact contains the model, crash harness, raw case outcomes, environment manifests, and shortest traces. A related four-author middleware manuscript and a contribution-overlap statement are supplied separately for chair assessment. The experiments here were newly constructed for this study; no implementation benchmarks or numerical results from that manuscript are reused.

AI Assistance Disclosure

OpenAI ChatGPT assisted in formulating the model, generating the experiment and validation code, and drafting the text in Sections I–VII. The reported outputs were produced by executing the supplied code. This disclosure does not replace author review of the model, implementation, or claims.

References

[1] NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.

[2] E. Barker et al., *Considerations for Achieving Crypto Agility: Strategies and Practices*, NIST CSWP 39upd1, June 2026, doi: 10.6028/NIST.CSWP.39-upd1.

[3] E. Barker, *Recommendation for Key Management: Part 1—General*, NIST SP 800-57 Part 1 Rev. 5, May 2020, doi: 10.6028/NIST.SP.800-57pt1r5.

[4] Amazon Web Services, "Rotate AWS KMS keys," AWS KMS Developer Guide. Accessed: Nov. 7, 2026. [Online]. Available: https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html

[5] MongoDB, "Rotate and rewrap encryption keys," MongoDB Manual. Accessed: Nov. 7, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/core/queryable-encryption/fundamentals/manage-keys/

[6] T. S. Pillai et al., "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in *Proc. USENIX OSDI*, 2014. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[7] SQLite, "Atomic commit in SQLite." Accessed: Nov. 7, 2026. [Online]. Available: https://sqlite.org/atomiccommit.html

[8] G. Alagic et al., *Recommendations for Key-Encapsulation Mechanisms*, NIST SP 800-227, Sept. 2025, doi: 10.6028/NIST.SP.800-227.